UNIVERSIDADE DE BRASÍLIA
INSTITUTO DE CIÊNCIAS EXATAS
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO

Nome:_____ **Matrícula:** _____
Disciplina: Estruturas de Dados

Instruções:

1. Leia atentamente todas as questões antes de começar a responder.
2. Utilize caneta azul ou preta para escrever suas respostas.
3. Escreva de forma clara e legível.
4. Responda apenas o que foi solicitado.
5. Verifique o tempo disponível e gerencie-o adequadamente.
6. Não é permitido o uso de dispositivos eletrônicos durante a prova.
7. Caso tenha alguma dúvida, levante a mão e aguarde a orientação do professor.
8. Boa prova!

1. (2.5 pontos) Uma fila é uma estrutura de dados do tipo FIFO (First-In, First-Out), ou seja, o primeiro elemento inserido é o primeiro a ser removido. Implemente os métodos **`enqueue`** e **`dequeue`** da classe **`Fila`**. A fila deve ser representada internamente por uma **`tupla`**. O método **`enqueue(elemento)`** recebe um elemento e o adiciona ao final da fila e não retorna nada. O método **`dequeue()`** remove e retorna o elemento no início da fila. Se a fila estiver vazia, ele deve retornar **`None`**.

```
class Fila:
    def __init__(self):
        self.fila = ()

    def enqueue(self, elemento):
        pass

    def dequeue(self):
        pass
```

2. (2.5 pontos) Considere uma lista simplesmente encadeada contendo uma referência para o primeiro nó/elemento, como aquela vista em aula. Considere que cada elemento desta lista armazena o nome de um site web e seu endereço na Internet (ambos são strings). Implemente o método **`buscar(site)`** que, dado o nome de um site, busque o seu endereço correspondente na lista e, ao mesmo tempo, mova o elemento que contém o nome buscado para o início da lista, de forma que buscas subsequentes ao mesmo site sejam mais rápidas. Caso o nome não esteja na lista, o método deve apenas retornar **`None`**.

```
class Node:
    def __init__(self, site, endereco):
        self.site = site
        self.endereco = endereco
        self.next = None

class SiteList:
    def __init__(self):
        self.head = None

    def adicionar(self, site, endereco):
        novo = Node(site, endereco)
        novo.next = self.head
        self.head = novo

    def buscar(self, site):
        pass
```

3. (2.5 pontos) Dado um vetor de tamanho *n*, mostre a complexidade do melhor, médio e pior caso da busca sequencial e busca binária para pesquisa com e sem sucesso. **JUSTIFIQUE!**

	Busca Sequencial			Busca Binária		
	melhor	médio	pior	melhor	médio	pior
Com sucesso						
Sem sucesso						

4. (2.5 pontos) Considere uma tabela de dispersão (*hash table*) de tamanho 10, utilizando o método de endereçamento aberto com sondagem linear (*rehash*) com pulo 1 para resolver colisões. A função de dispersão (*hash function*) usada é `h(x) = x % 10`. Dada a sequência de chaves `23, 43, 13, 27, 33, 37` para inserção na tabela, faça:

Insira cada uma das chaves em ordem na tabela de dispersão, mostrando o estado da tabela após cada inserção. Indique a posição de cada chave na tabela e explique como a sondagem linear é aplicada em caso de colisão.